初创公司的FinOps之路：两年内云成本节省80%，无需专职团队！

李国银 InfoQ 2024年09月23日 01:01



作者 | 李国银

> 阅读完本文耗时将大于 20 分钟，作者总结了较大篇幅的实战经验，非常有收藏价值，初次阅读建议您可以先跳过该章节（已标记 TL;NR. ）
> 标题里的这家公司就是杭州云猿生数据，这家公司在近两年，使用 FinOps 工具、流程和策略，云资源节省了 80% 以上成本。

在取得该成效的过程中，和大公司不同的地方，没有投入一个专门团队做所谓“降本增效”战役，也未高价购入 FinOps 产品或者引入专业的成本优化供应商，完全以内部研发团队兴趣小组的方式，自建共享迭代优化，且在 FinOps 过程中以零打搅研发效率为目标。

如您也希望如此，可以展开阅读以下。

什么是 FinOps，为什么企业需要 FinOps?

FinOps（Financial Operations）是一种云财务管理实践，旨在帮助企业优化其云计算支出，同时提高业务敏捷性和创新能力。随着云服务的普及，企业在云上的支出日益增长，而 FinOps 通过跨部门协作（如财务、技术和业务团队）来管理和优化这些支出。

FinOps 的核心目标包括：

1. **可见性**：提供对云支出的透明度，让不同团队了解自己的使用情况和成本。
2. **优化**：通过技术、财务和流程上的改进来减少不必要的支出，提高成本效益。
3. **协作**：财务团队和技术团队共同制定和实施成本管理策略。

FinOps 的关键原则：

1. **团队所有权**：每个团队对自己的云使用和成本负责。
2. **速度与成本平衡**：在控制成本的同时，不牺牲业务的速度和创新。
3. **持续改进**：随着云环境和需求的变化，持续优化和调整支出。

FinOps 结合了技术和财务的最佳实践，使组织能够更好地预测、控制和优化其云资源的使用。

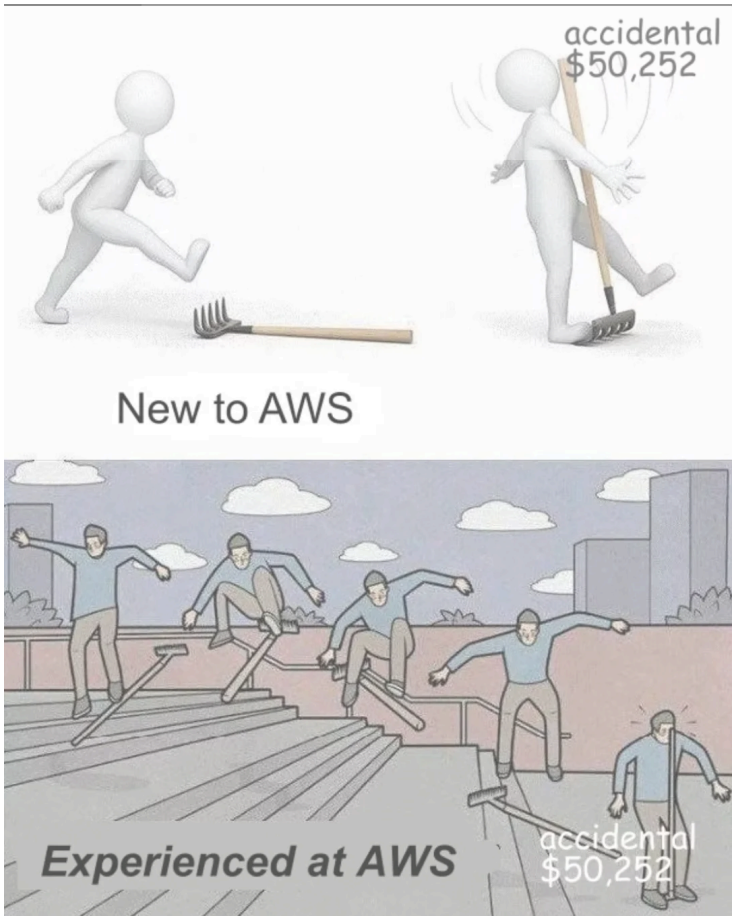
以 NBA 比赛为例，如果说企业的核心业务是 NBA 得分球星迈克乔丹，那么 FinOps 就是让企业在商业竞争中能撑得更久的 NBA 篮板王大虫罗德曼。

如何在企业从零到一做好 FinOps 工作?

本文的目标读者，是需要在企业从零到一做好 FinOps 工作的负责人以及相关干系人。

基于云猿生 2 年从零到一的 FinOps 实践经验看，将多云管理从:"无度量"、"失控"的初始状态，调协到:"全颗粒度"、"充分掌控"的状态，达成将用云成本降低 50% 以上的 KPI 目标，是完全可行的。

在您阅读本文之前，或许您已经了解到 FinOps 是一个系统工程，虽然关于 FinOps 的各类技术分享、网络分享已经汗牛充栋，但是仔细阅读这些文章，您会发现大多数都是只讲述一个简单的点，没有系统的、实操性质的可落地企业一线并直接带来重要改变的方案。因此当您阅读完这些文章之后，大部分情况你只是知道了这一个小知识，而与此同时，您在线上运行的各种云资源并未因为您的知识增长而对您有所畏惧，依然无情地吞噬着企业极为珍贵的资金。



本文将会基于这个目标展开：如何在企业从零到一做好 FinOps 工作？让您真正且快速成为一个拯救企业命运的英雄，即使您对 FinOps 现在一无所知。

云猿生的用云基本情况

麻雀虽小，五脏俱全。云猿生虽然是一家初创公司，但是用云范围几乎已全面覆盖市场上的主流云，以及拥有私有化的 IDC 机房和自建的 Lab 机房。



云猿生使用云原生技术对于负载的调度，比如 K8s, Terraform，Karpenter。

如果贵司用云的范式也是如此，那么您应该能体会多云管理和成本优化的复杂度。

云猿生的用云基本原则

和大企业不一样，云猿生是一家硅谷风格的科技创业公司，崇尚工程师文化和效能，因此用云的基本原则是：不审批，开放用云。

基于这个用云基本原则，极大地减少了用云管理成本，并且让全员都解锁了用云知识，使得云猿生公司快速具备能够处理多云使用和管理复杂度的人才广度。也基于此，云猿生推出多云混合云数据库管理平台，基于该平台，用户可自主选择公有云提供商的实例折扣计划，让云数据库的成本最多节省 75%。

在关于 FinOps 的各类技术分享中，很少考虑这个用云的管理成本，它们其实就是在玩转移复杂度的游戏，只减肚子上的肥肉而不考虑内脏里的脂肪肝。云猿生在“不审批，开放用云”的基本原则上，执行 FinOps 同时也用技术和更简单开放透明的管理方式做到几乎为零的管理成本。

- 技术管理方式比如：使用多云统一登录解决方案。

使用多云统一登录，使得全体员工可以无需逐一申请创建每个云的 IAM User 就可以直接一键登录访问任意云，并且可以更方便进行用云审计和自动化云资源分账，兼顾了成本、效率和合规。

- 更简单开放透明的管理方式比如：

1. 制定《用云规约》，让全体员工在规约的基础上使用云。

2. 《用云规约》公告了成本优选的用云顺序、赠金额度情况、折扣和优惠、通用约定规则、以及每个云的特殊规则。基于规约，即使是新员工也可以做到无人指导的情况下像 FinOps 专家一样找到划算的用云方式。

对以上管理方式有兴趣的客户，我司可以提供免费技术指导。

如何自动化的落地 FinOps?

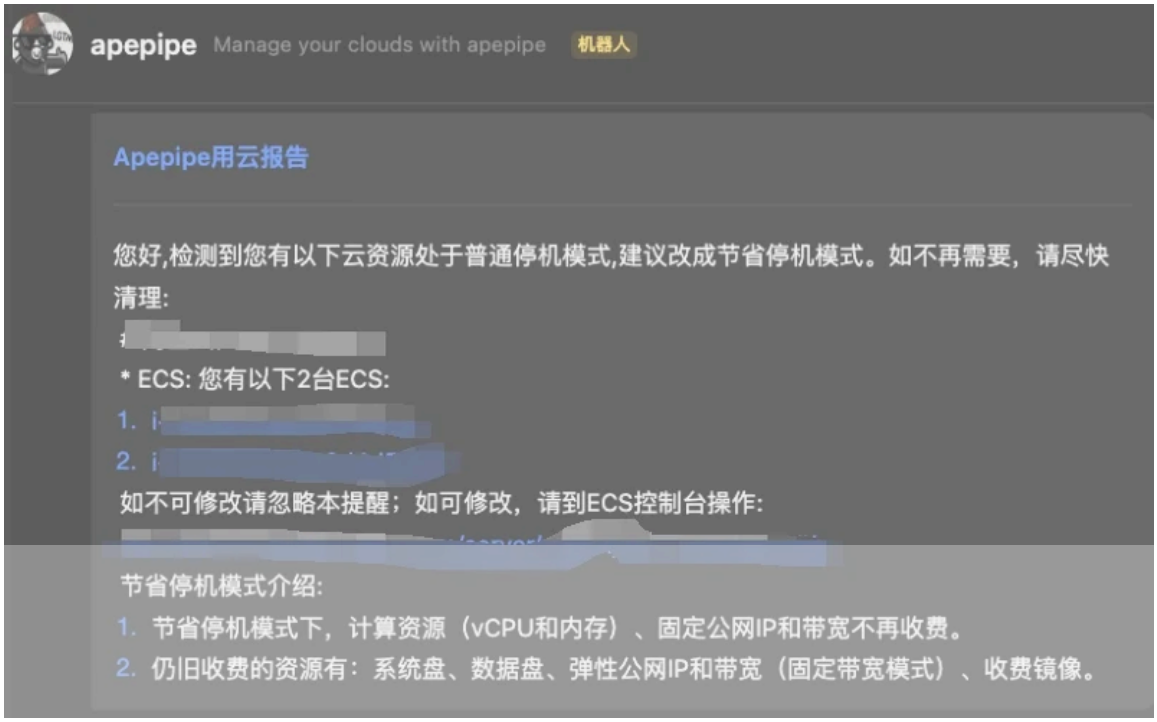
拥有原则并不能保证 FinOps 的落地。FinOps 是一个持续性的工作，特别是基于我司用云范式的复杂度，没有工具只靠各个云的管控平台，已无法保障。

因此云猿生内部开发了一个 FinOps 副产品：Apepipe。实现对多云多账号的统一 FinOps 自动化。目前该产品已经有以下能力，并可以私有化部署：

Apepipe Bot

通过飞书和 Slack 等机器人（有需要也可以支持钉钉、企业微信等），实现 FinOps 功能的交互。

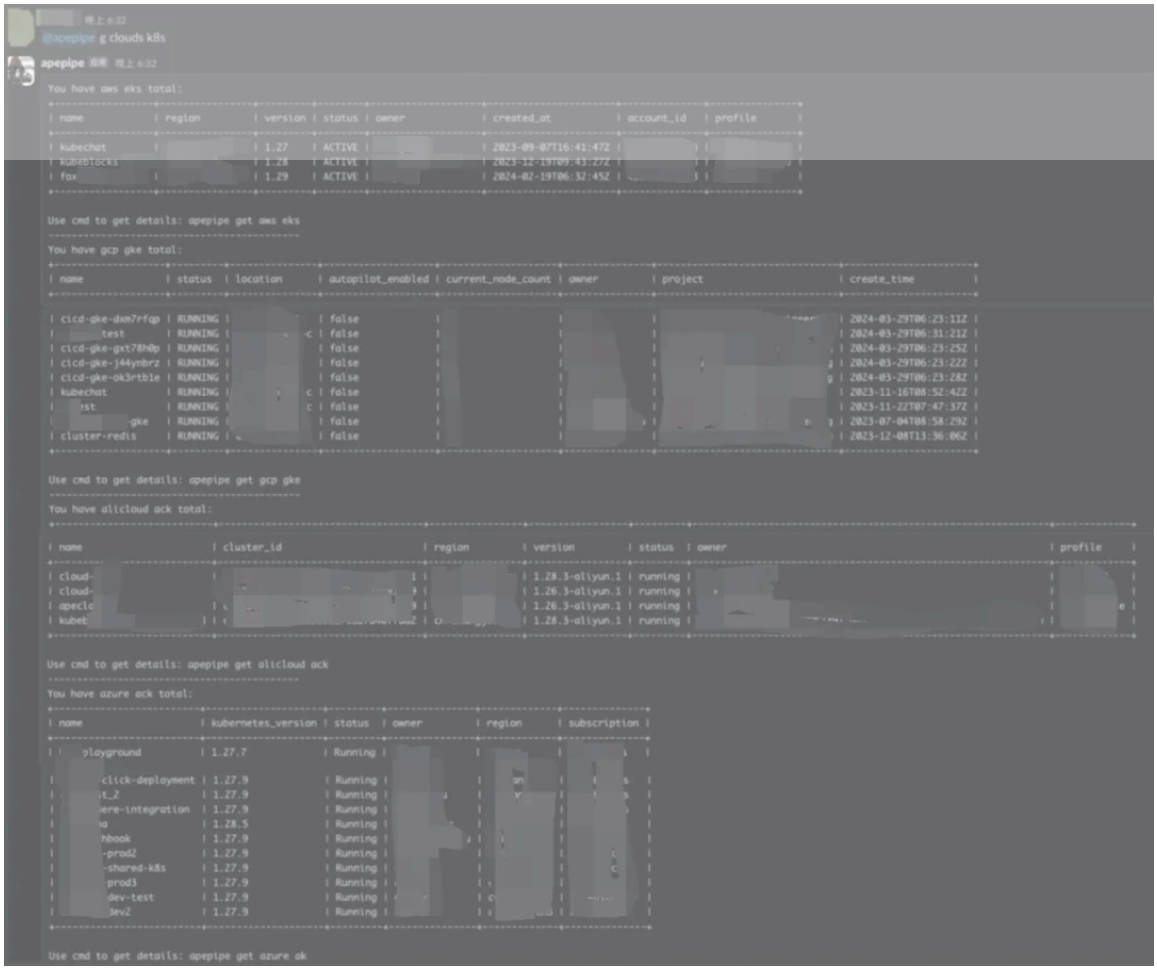
比如推送 FinOps 账单和优化提醒：



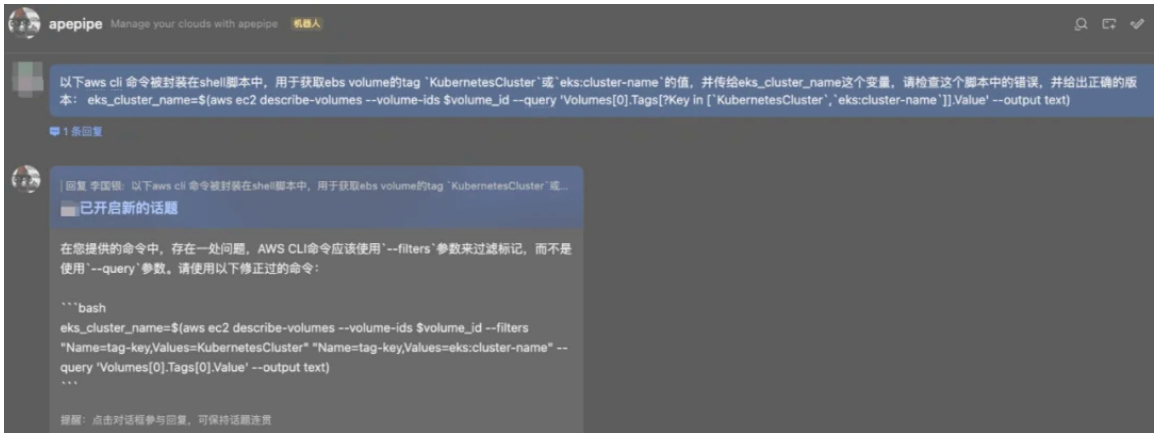


Apepipe Bot 也可以用 ChatOps 的方式调用 CLI 的所有命令，比如查看云资源、删除云资源、或查实例的价格，这样当你躺在沙滩上时，随便打开飞书、Slack 等就可以轻松管理所有云的云资源了：





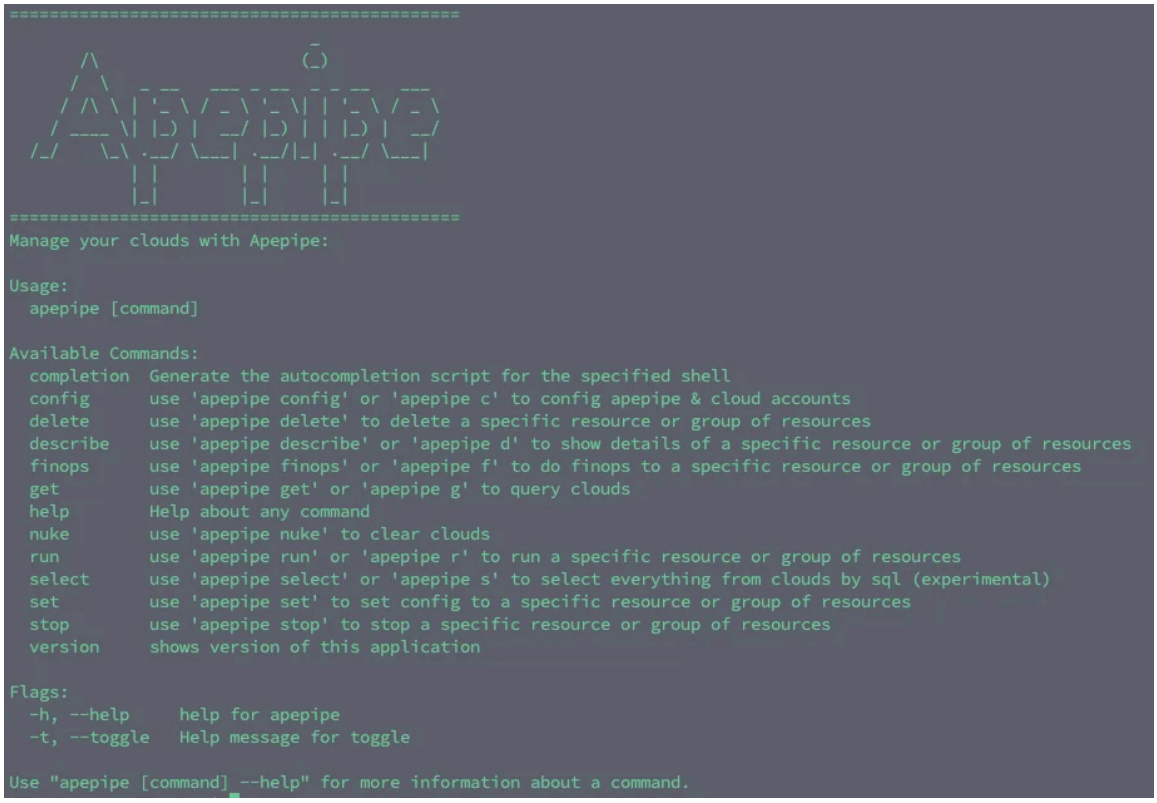
- Apepipe Bot 接入现在流行的 AI，FinOps 起来就更趁手了：





Apepipe CLI

通过 CLI 可以在命令行下做统一多云资源查询和管理。

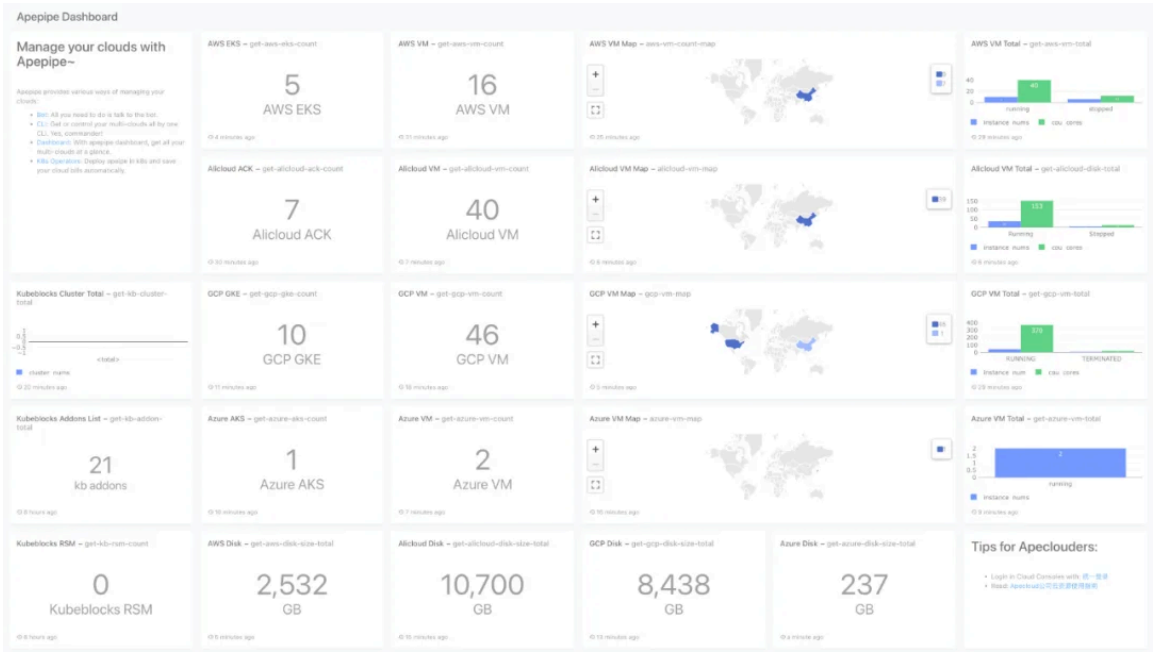


Apepipe CLI 还集成了一些效能工具，比如当你不再需要使用这个云账号时，使用 nuke 这个工具可以把账号里的资源一键清理干净。

Apepipe Dashboard

通过 dashboard 可以大屏一览所有云资源的情况，也可以详细定位查询某个云资源，无需在多云多账号中切换。

零代码可以自定义的 Dashboard:



Apepipe Operators

部署在 k8s 中，按 FinOps 规则进行自动资源优化的执行器。FinOps 最大的敌人是时间，只要资源在那里，云账单每分每秒都在增加计费，这些执行器可以在后台持续运行，无需人工介入，在每分每秒为你消除浪费的云资源，并按规则合理地优化资源。比如：

1. 自动清理掉不用的磁盘
2. 自动将 AWS GP2 转为 GP3
3. 自动优化 AWS EC2 实例的 credit 设置
4. 自动在周末关机测试用的云主机
5. 自动清理不再使用的日志项目

```
NAMESPACE   NAME                                        COMPLETIONS   DURATION   AGE
apepipe     change-instance-credit-to-standard-28771680   1/1           12s        17m
apepipe     change-volume-type-to-gp3-28771690            1/1           7s         7m11s
apepipe     delete-available-volume-28771680              1/1           7s         17m
apepipe     stop-ec2-28770719                             1/1           14s        16h
```

云猿生 FinOps 实战经验（TL;NR）

限于公众号篇幅，以下筛选了一些常见的 FinOps 实战，更多的分享总结在《FinOps Zero 2 Hero》电子书中，将在之后整理公开分享出来。

云服务器成本

1. 1 使用 Spot 实例节省 50～90% 成本

对于开发测试非稳定性要求的需求，建议使用 Spot 实例节省云服务器成本。

其中，GCP 的 spot 实例可以只停止不删除，特别推荐使用。（AWS 的 spot 实例当被平台回收时，实例绑定的系统盘也会一并被回收，如果盘上有数据，将无法找回）

GCP spot 实例成本是 standard 的 1/3

1.2 长驻需求的云服务器使用预留实例

预留实例选用的技巧：

相关变量：1. 是否预付费；2. 是否可变；3. 周期（Term）；4. 范围（Scope）

说明：

1. 是否预付费： 选预付费就是提前把钱支付掉；如果选用不预付费的方式结算，月底和供应商拉账单月结；预付费的价格会更低；
2. 是否可变：可变是指在预定周期内，可以变化实例类型。不可变的叫标准模式（Standard）。
3. 周期（Term）：预定的时长，一般有 1 年和 3 年；3 年便宜很多。
4. 范围（Scope）：Region 和 Zone，Zone 的一般更便宜。

如果确定性强的需求，可以选用：预付费 + 不可变 +3 年，这样成本是最低的。

1.3 在相互不干扰的情况下，尽可能合用资源

1.4 使用定时任务，配置 VM，晚上不用时自动关机，早上使用时自动开机，节省费用

使用定时任务，也可以设置周末定时停机

1.5 暂时不用的实例，打成机器镜像

当实例暂时不用的时间超过 1 周或者当实例的块存储的费用比较高时，相比于仅停机，打成机器镜像然后删除实例，可以节省块存储费用。需要的时候，用机器镜像恢复就可以了

1.6 节省停机模式

阿里云的停机方式有两种，普通停机模式，成本还是一样在扣减的，建议当云服务器暂时不用时，选用节省停机模式。

节省停机模式介绍:

1. 节省停机模式下，计算资源（vCPU 和内存）、固定公网 IP 和带宽不再收费。
2. 仍旧收费的资源有：系统盘、数据盘、弹性公网 IP 和带宽（固定带宽模式）、收费镜像。

1.7 可用区也会影响成本

选可用区的小技巧：三短一长选长的，三长一短选短的；两长两短选择 B，长短不齐就选 BC。有一种“民间偏方”叫做选 C。

这个小技巧，特别对于使用 spot 的实例的时候效果明显。在 2 年的 AWS，GCP 等云的 Spot 实例回收概率分析后，A 可用区的回收概率远远高于 B,C 和之后的其他可用区。

因此，想要省钱，可用区的选择也是有讲究的哦。

1.8 Spot 实例的回收消息里的宝藏

Spot 实例的回收是一个概率事件，通过分析不同实例类型、不同可用区的回收概率，可以优化调整 Spot 的部署，就可能用 10% 的成本（Spot）来达成需求的同时有很高的稳定性。



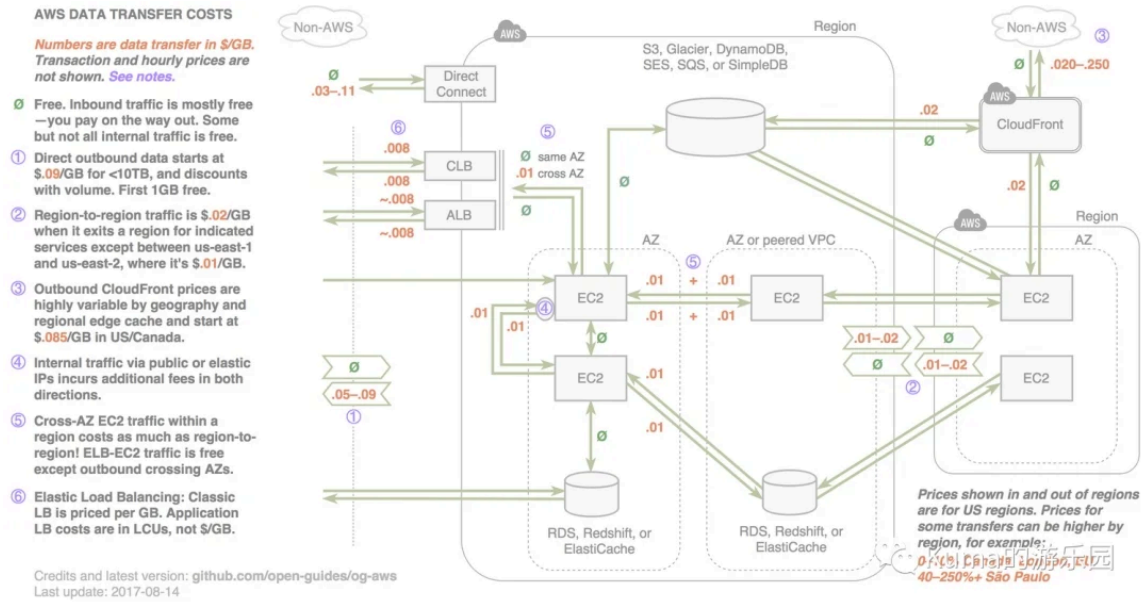
2. 公网 IP 成本

目前几乎所有云的公网 IP 都已经单独计费，因此，对于可以不用公网访问的情况，不配置公网 IP。减少公网 IP 使用的另一个好处是，减少被公网恶意流量攻击的可能。

3. 流量成本

3.1 AWS 在同一个 region 内，不同 az 的 EC2 之间的网络流量，等同于跨 region，也是需要同样的流量成本

真实案例：某公司项目组因为不了解该信息，多 AZ 的 EC2 之间几天跑出了 7 万多的流量成本。



3.2 流量尽量走内部，公网流量很贵

比如：阿里云 ECS 读写 OSS，一个 1TB 的 bucket，如果走公网 endpoint，流量 0.25 元/GB，读完这个 bucket，250 块就没了（数字举例纯属精确且巧合）。

4. 对象存储成本

4.1 S3: 如何正确删除一个 S3 bucket，当文件数量大到无法手工清理的时候

我们经常会遇到测完 S3 后面要删数据，bucket 删除前会提示先删除里面的数据，当文件数量大到无法手工清理的时候，怎么办?

这么多大文件，需要用 lifecycle 来删除，不要直接 delete，不然删除也会产生较大的流量费用。

```
1 YAML
```

```
s3api put-bucket-lifecycle --bucket %s --lifecycle-configuration '{
"Rules": [
    {
        "ID": "delete-all",
        "Status": "Enabled",
        "Prefix": "",
        "Expiration": {
            "Days": 1
        }
    }
  ]
}'
```

4.2 阿里云 OSS：用 ZRS 会比 LRS 贵 25%，开发测试没有特殊需求的话，建议用 LRS



4.3 定时清理不再使用的 bucket

按量计费的对象存储，随着时间的累计，也是一笔不小的费用。应该定时清理不再使用的 bucket。

不清理出了存储本身的成本，更危险的是：公有云上的黑暗森林法则出现了：只要你的 S3 对象存储桶名暴露，任何人都有能力刷爆你的云账单。

https://medium.com/@maciej.pocwierz/how-an-empty-s3-bucket-can-make-your-aws-bill-explode-934a383cb8b1

5. 块存储成本

5.1 停止的实例，可以把块存储转成快照节省成本

5.2 阿里云块存储的选用：performance_level PL1 要比 PL0 贵一倍

阿里云 ACK 默认的 essd 的 SC 会选用 PL1，一般非性能测试需求的开发测试，PL0 的性能已经足够用了。阿里云 ACK 可以新配置一个 essd 的 SC，加上 performanceLevel: PL0 的参数。

6. GPU 成本

6.1 使用“平替”的 GPU 型号

Nvidia L4 机型的成本比 A100 便宜非常多，但 Cap 甚至高于 A100，对于带宽要求不高的情况，可以考虑作为平替。

	A	B	C	D	E
1	GPU	Machine Type	Price - USD(Monthly)	Price - RMB(Monthly)	
2	NVIDIA L4	g2-standard-4	526	3839.8	
3	A100 40G	a2-highgpu-1g	2692	19651.6	
4	A100 80G	a2-ultragpu-1g	3710	27083	
5					
6					
7					

	A	B	C
1	section	GPU	Compute Capability
2	Tesla Workstation Products	Tesla K80	3.7
3		Tesla K40	3.5
4		Tesla K20	3.5
5		Tesla C2075	2
6		Tesla C2050/C2070	2
7	NVIDIA Data Center Products	NVIDIA H100	9
8		NVIDIA L4	8.9
9		NVIDIA L40	8.9
10		NVIDIA A100	8
11		NVIDIA A40	8.6
12		NVIDIA A30	8
13		NVIDIA A10	8.6
14		NVIDIA A16	8.6
15		NVIDIA A2	8.6
16		NVIDIA T4	7.5
17		NVIDIA V100	7
18		Tesla P100	6
19		Tesla P40	6.1
20		Tesla P4	6.1
21		Tesla M60	5.2
22		Tesla M40	5.2
23		Tesla K80	3.7
24		Tesla K40	3.5
25		Tesla K20	3.5
26		Tesla K10	3

6.2 GCP 巧妙解决项目 GPU 的 quota 限制

GCP 的 billing 是基于项目维度的，不同的项目可以绑定不同的支付方式。GCP 的 quota 和支付方有关系，比如某公司 GCP 账号的供应商支付的 A100 GPU quota 有 16 个， 为了获得这个 quota 的同时并且用 GCP 初创公司赠金，操作如下：

```
1 新建一个项目，通过先绑定到供应商billing获得16个quota，然后切到自己的赠金billing
```

新建一个项目，通过先绑定到供应商 billing 获得 16 个 quota，然后切到自己的赠金 billing另外，on-demand 和 spot 的 quota 都是独立的，在一类 quota 满的时候，可以试试换一类使用。

7. 快照成本

7.1 快照存档

当快照需要长期保存（>90 days），将快照进行归档可以节省 75% 的成本，参考：*https://docs.aws.amazon.com/ebs/latest/userguide/snapshot-archive.html*

8. 日志成本

8.1 关闭不需要的日志生成

比如 eks, gke 等云托管的 k8s 都会默认生成日志；云上的日志系统，无论是 AWS 的 cloudwatch，还是 GCP 的 logs storage，日志一开，费用就刷刷去了。比如某公司项目 gke 因为大量刷 error log 产生了 10 个 T 的日志，消耗了 6,877.65 美金。

8.2 缩短日志存储周期

日志的每 GB 存储费用远高于对象存储。因此尽可能减短 Retention period，以 GCP log storage 为例，最短可以配置成 1 天。

9. 容器服务成本

9.1 eks 需要保持 k8s 版本升级，不然扩展支持需要支付额外费用

10. AWS 专区

12.1 积分规范

积分规范：t3 /t4g 支持 cpu 突发，比如核数跑满的话，会额外消耗 cpu credit，动用额外的核来支持计算，这个超出自己账号省出来的 cpu credit 后，会额外收费，0.342 CNY

per vCPU-Hour of T3 CPU Credits，因此建议普通开发测试需求，设置“积分规范”的模式为"standard"，这样可以避免额外收费。

12.2 KMS 的成本优化

使用 terraform 创建 EKS，当销毁时，和 eks 一起创建的 KMS key 的默认删除周期是 30 天，在 terraform 配置里加上下面这句配置，将可以节省 KMS key 在等待删除周期里的 75% 的成本：

```
YAML
deletion_window_in_days = 7
```

13. GCP 专区

13.1 建议选择 us-central1（VM 价格最便宜），可以咨询 Google 官方了解价格表：

- *https://cloud.google.com/compute/all-pricing*
- *https://www.instance-pricing.com/provider=gcp/cheapest/*
- *https://googlecloudplatform.github.io/region-picker/*

13.2 在 GKE 上运行费用经过优化的 Kubernetes 应用的最佳做法：

- *https://cloud.google.com/architecture/best-practices-for-running-cost-effective-kubernetes-applications-on-gke?hl=zh-cn#spread_the_cost-saving_culture*

13.3 整理一份 GCP 实例规格，并标注了 FinOps 推荐的机型



	E2 (finops推荐使用)	N2	N2D	T2D	T2A	C2	C2D	M2	M3	N1+GPU	A2	N1 (finops不推荐使用)	M1(finops不推荐使用)
工作负载类型	费用优化	通用	通用	横向扩容优化通用	横向扩容优化通用	计算优化	计算优化	内存优化	内存优化	加速器优化	加速器优化	通用	内存优化
CPU 类型	Skylake、Broadwell、Haswell、AMD EPYC Rome 和 AMD EPYC Milan	Cascade Lake 和 Ice Lake	AMD EPYC Rome 和 AMD EPYC Milan	AMD EPYC Milan	Ampere Altra	Cascade Lake	AMD EPYC Milan	Cascade Lake	Ice Lake	Skylake、Broadwell、Haswell、Sandy Bridge 和 Ivy Bridge	Cascade Lake	Skylake、Broadwell、Haswell、Sandy Bridge 和 Ivy Bridge	Skylake 和 Broadwell
架构	x86	x86	x86	x86	Arm	x86	x86	x86	x86	x86	x86	x86	x86
vCPU 数	0.25 到 32	2 到 128	2 到 224	1 到 60	1 到 48	4 到 60	2 到 112	208 到 416	32 到 128	1 到 96	12 到 96	1 到 96	40 到 160
vCPU 定义	线程	线程	线程	核心	核心	线程	线程	会话	会话	线程	线程	线程	线程
内存大小	1 到 128 GB	2 到 864 GB	2 到 896 GB	4 到 240 GB	4 到 192 GB	16 到 240 GB	4 到 896 GB	5888 到 11776 GB	976 到 3904 GB	3.75 到 624 GB	85 到 1360 GB	1.8 到 624 GB	961 到 3844 GB
扩展内存	–	✅	✅	–	–	–	–	–	–	✅	–	✅	–
单租户	–	✅	✅	–	–	✅	✅	✅	✅	✅	–	✅	✅
嵌套虚拟化	–	✅	–	–	–	–	–	–	–	✅	–	✅	–
自定义虚拟机形式	✅	✅	✅	–	–	–	–	–	–	✅	–	✅	–
机密计算	–	–	–	–	–	–	–	–	–	–	–	–	–
磁盘接口类型	SCSI 和 NVMe	SCSI 和 NVMe	SCSI 和 NVMe	SCSI 和 NVMe	NVMe	SCSI 和 NVMe	SCSI 和 NVMe	SCSI 和 NVMe	NVMe	SCSI 和 NVMe	SCSI 和 NVMe	SCSI 和 NVMe	SCSI 和 NVMe
本地固态硬盘 (SSD)	–	✅	✅	–	–	✅	✅	–	–	✅	✅	✅	✅
最大本地 SSD	0	9 TB	9 TB	0	0	3 TB	3 TB	0	0	9 TB	3 TB	9 TB	3 TB
标准永久性磁盘	可用区和区域	可用区和区域	可用区和区域	可用区	可用区	可用区	可用区	可用区级	可用区级	可用区和区域	可用区	可用区和区域	可用区
平衡永久性磁盘	可用区和区域	可用区和区域	可用区和区域	可用区	可用区	可用区	可用区	可用区级	可用区级	可用区和区域	可用区	可用区和区域	可用区
SSD 永久性磁盘	可用区和区域	可用区和区域	可用区和区域	可用区	可用区	可用区	可用区	可用区级	可用区级	可用区和区域	可用区	可用区和区域	可用区
极端永久性磁盘	–	✅	–	–	–	–	–	✅	✅	–	–	–	✅
网络接口	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net	gVNIC 和 VirtIO-Net
网络性能	1 到 16 Gbps	10 到 32 Gbps	10 到 32 Gbps	10 到 32 Gbps	10 到 32 Gbps	10 到 32 Gbps	10 到 32 Gbps	32 Gbps	32 Gbps	2 到 32 Gbps	24 到 100 Gbps	2 到 32 Gbps	32 Gbps
高带宽网络	–	50 到 100 Gbps	50 到 100 Gbps	–	–	50 到 100 Gbps	50 到 100 Gbps	–	是	–	50 到 100 Gbps	–	–
GPU 数上限	0	0	0	0	0	0	0	0	0	8	16	0	0
折扣	CUD 和 Spot	SUD、CUD 和 Spot	SUD、CUD 和 Spot	CUD 和 Spot	Spot	SUD、CUD 和 Spot	SUD、CUD 和 Spot	SUD 和 CUD	SUD 和 CUD	SUD、CUD 和 Spot	CUD 和 Spot	SUD、CUD 和 Spot	SUD、CUD 和 Spot
Coremark 得分 (1vCPU)	1.04	1.28	1.46	2.29	–	1.43	1.5	1	–	–	–	1	0.96

source: https://cloud.google.com/compute/docs/machine-types?hl=zh-cn
edited by apecloud,2023-9-4

14. 拥有自己的数据中心和私有云

为了提高稳定性和备份能力，云猿生选择了多云架构，目前已支持阿里云、AWS、Azure、GCP、电信云、腾讯云、百度云、火山引擎，并与 SealOS 进行了合作；与此同时云猿生也有私有云的投资，尽管私有云的机器部署要比公有云更为繁琐，但我们认为拥有自己的数据中心和私有云是一项长期的投资决策，可以降低每年的云服务费用。

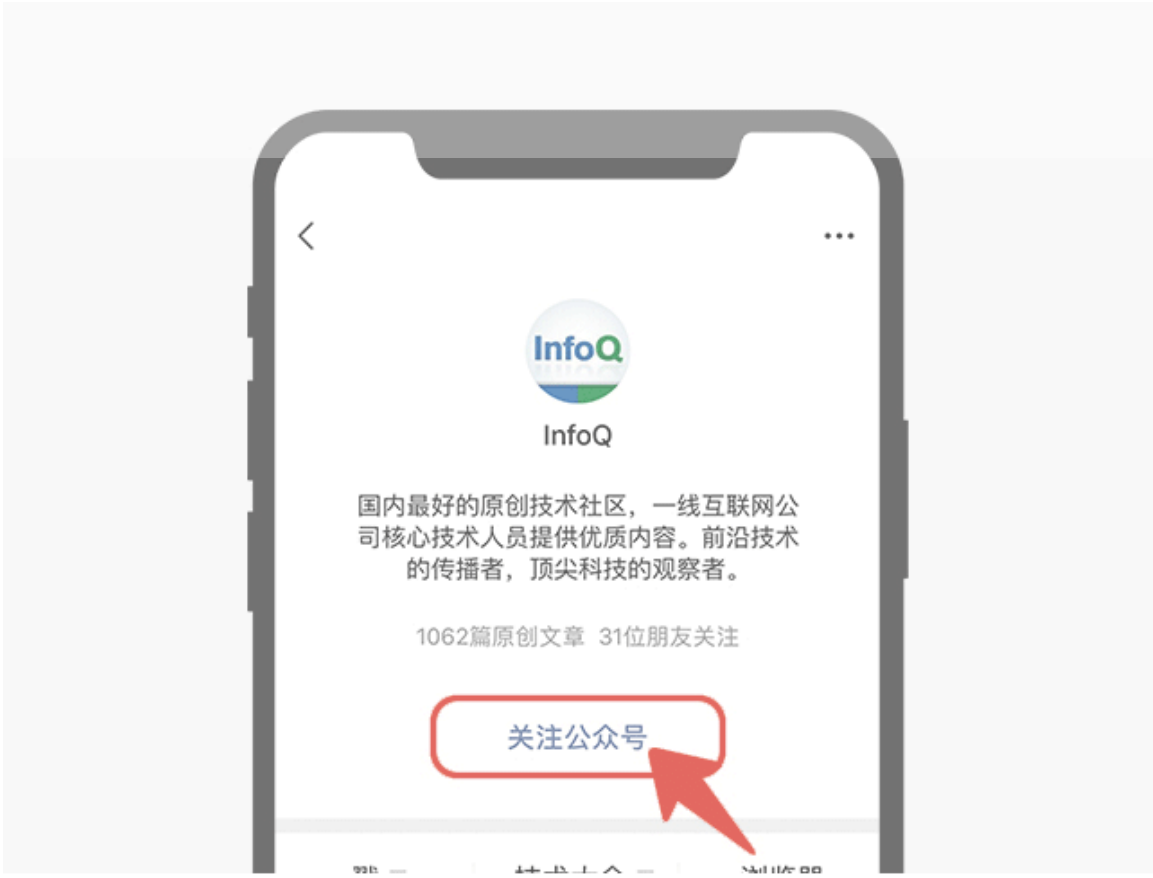
今日好文推荐

下载量超 5000 万的知名应用，开发团队“全军覆没”，从此发版人唯剩老板一个

突发！高通被曝有意收购英特尔；思科N+7裁员，员工称人性化；百度最高奖发出2800万！李彦宏：再苦不能苦技术人 | Q资讯

内部数百工程师可随意摄取 OpenAI 先进模型！OpenAI 前员工揭露：谏言即被开除，祈祷公司不报复

JavaScript 之父联手近万名开发者集体讨伐 Oracle：给 JavaScript 一条活路吧！



InfoQ

前沿技术的传播者
顶尖科技的观察者